Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms
Chapter 5. Dynamic Programming
Session 1

by:

Er. Bikal Adhikari

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

Chapter Outline

Chapter 5. Dynamic Programming

8 hours

5.1 Basic concepts
5.2 All pair shortest path algorithm
5.3 Travelling salesperson problem
5.4 String editing
5.5 0/1 knapsack problem using dynamic programming
5.6 Matrix chain multiplication
5.7 Flow shop scheduling

Review of Matrix Multiplication

Matrix Multiplication

$A=(a_{ij})_{m1\times n1}$, $B=(b_{ij})_{m2\times n2}$

Required condition: n1 = m2

$C=(c_{ij})_{m1\times n2}$

where, c_{ij} is obtained by getting sum product of ith row of matrix A with j^{th} column of matrix B. We find c_{ij} for every i,j

$$A=\begin{pmatrix} a[0][0] & a[0][1] & a[0][2] \\ a[1][0] & a[1][1] & a[1][2] \\ a[2][0] & a[2][1] & a[2][2] \end{pmatrix}_{3\times 3}, B=\begin{pmatrix} b[0][0] & b[0][1] & b[0][2] \\ b[1][0] & b[1][1] & b[1][2] \\ b[2][0] & b[2][1] & b[2][2] \end{pmatrix}_{3\times 3}$$

$$C=\begin{pmatrix} c[0][0] & c[0][1] & c[0][2] \\ c[1][0] & c[1][1] & c[1][2] \\ c[2][0] & c[2][1] & c[2][2] \end{pmatrix}_{3\times 3}$$

$$c[0][0]=a[0][0]*b[0][0]+a[0][1]*b[1][0]+a[0][2]*b[2][0]$$

Matrix Multiplication

$A=(a_{ij})_{m1\times n1}$, $B=(b_{ij})_{m2\times n2}$

Required condition: n1 = m2

$C=(c_{ij})_{m1\times n2}$

where, c_{ij} is obtained by getting sum product of ith row of matrix A with j^{th} column of matrix B. We find c_{ij} for every i,j

$$A=\begin{pmatrix} a[0][0] & a[0][1] & a[0][2] \\ a[1][0] & a[1][1] & a[1][2] \\ a[2][0] & a[2][1] & a[2][2] \end{pmatrix}_{3\times 3}, B=\begin{pmatrix} b[0][0] & b[0][1] & b[0][2] \\ b[1][0] & b[1][1] & b[1][2] \\ b[2][0] & b[2][1] & b[2][2] \end{pmatrix}_{3\times 3}$$

$$C=\begin{pmatrix} c[0][0] & c[0][1] & c[0][2] \\ c[1][0] & c[1][1] & c[1][2] \\ c[2][0] & c[2][1] & c[2][2] \end{pmatrix}_{3\times 3}$$

$$c[0][1]=a[0][0]*b[0][1]+a[0][1]*b[1][1]+a[0][2]*b[2][1]$$

Matrix Multiplication

$A=(a_{ij})_{m1\times n1}$, $B=(b_{ij})_{m2\times n2}$

Required condition: n1 = m2

$C=(c_{ij})_{m1\times n2}$

where, c_{ij} is obtained by getting sum product of ith row of matrix A with j^{th} column of matrix B. We find c_{ij} for every i,j

$$A=\begin{pmatrix} a[0][0] & a[0][1] & a[0][2] \\ a[1][0] & a[1][1] & a[1][2] \\ a[2][0] & a[2][1] & a[2][2] \end{pmatrix}_{3\times 3}, B=\begin{pmatrix} b[0][0] & b[0][1] & b[0][2] \\ b[1][0] & b[1][1] & b[1][2] \\ b[2][0] & b[2][1] & b[2][2] \end{pmatrix}_{3\times 3}$$

$$C=\begin{pmatrix} c[0][0] & c[0][1] & c[0][2] \\ c[1][0] & c[1][1] & c[1][2] \\ c[2][0] & c[2][1] & c[2][2] \end{pmatrix}_{3\times 3}$$

$$c[2][1]=a[2][0]*b[0][1]+a[2][1]*b[1][1]+a[2][2]*b[2][1]$$

Tracing the Code for Matrix Multiplication

In general, the code for implementing matrix multiplication of two 3×3 matrices is as follows:

```
for(i=0;i<3;i++)
{
    for(j=0;j<3;j++)
    {
    sum=0;
        for(k=0;k<3;k++)
        {
        sum=sum+a[i][k]*b[k][j];
        }
    c[i][j]=sum;
    }
}
```

Note:

We have assumed array a stores first matrix, array b stores second matrix and array c stores product matrix

Tracing the Code for Matrix Multiplication

```
For any i and j
sum ← 0
k = 0
sum = sum +a[i][k]*b[k][j]
```




```
k = 1
sum = sum +a[i][k]*b[k][j]

k = 2
sum = sum +a[i][k]*b[k][j]

c[i][j]←sum
```

Tracing the Code for Matrix Multiplication

```
For any i and j
sum ← 0
k = 0
sum = sum +a[i][0]*b[0][j]
      0

k = 1
sum = sum +a[i][1]*b[1][j]

k = 2
sum = sum +a[i][2]*b[2][j]

c[i][j]←sum
```

Tracing the Code for Matrix Multiplication

```
For i=0,j=0
sum ← 0
k = 0
sum = sum +a[0][0]*b[0][0]
```




```
k = 1
sum = sum +a[0][1]*b[1][0]

k = 2
sum = sum +a[0][2]*b[2][0]

c[0][0]←sum
```

Tracing the Code for Matrix Multiplication

```
For i=0,j=1
sum ← 0
k = 0
sum = sum +a[0][0]*b[0][1]
```




```
k = 1
sum = sum +a[0][1]*b[1][1]

k = 2
sum = sum +a[0][2]*b[2][1]

c[0][1]←sum
```

Tracing the Code for Matrix Multiplication

```
For i=2,j=1
sum ← 0
k = 0
sum = sum +a[2][0]*b[0][1]
```




```
k = 1
sum = sum +a[2][1]*b[1][1]

k = 2
sum = sum +a[2][2]*b[2][1]

c[2][1]←sum
```

Dynamic Programming

→ Dynamic programming is the most powerful design technique for solving optimization problem.

→ Divide and conquer algorithms partition the problem into disjoint sub-problems solves the sub-problems recursively and then combine their solutions to solve the original problems.

→ Dynamic programming is used when the sub-problems are not independent, eg when they share the same sub-problem.

→ In this case, divide and conquer may do more work than necessary, because it solves the same sub problem multiple times.

Greedy Alg. Vs Dynamic Programming

SN	Greedy Algorithm	Dynamic Programming
1	Makes a locally optimal choice at each step hoping to obtain a global optimum.	Breaks the problem into smaller subproblems and combines their optimal solutions to obtain a global optimum.
2	Requires Greedy Choice Property and Optimal Substructure.	Requires Optimal Substructure.
3	Decisions are irrevocable (once made, they are not reconsidered).	Decisions are derived from stored optimal solutions of smaller subproblems.
4	Guarantees global optimum only if the greedy choice leads to an optimal solution (i.e., greedy choice property holds).	Guarantees global optimum if optimal substructure exists.
5	Does not systematically explore all subproblem combinations.	Systematically solves all necessary subproblems.

6	Not required.	Not required for correctness, but improves efficiency significantly.
7	Generally uses little or no extra memory.	Requires additional memory (memoization or tabulation).
8	Usually lower computational overhead (often involves sorting or simple iteration).	May have higher time and space complexity but reduces exponential recursion to polynomial time in many problems.
9	Easier to design and implement.	More complex; requires defining states and recurrence relations.
10	Fractional Knapsack, Activity Selection, Kruskal's Algorithm, Prim's Algorithm, Huffman Coding	0/1 Knapsack, Matrix Chain Multiplication, Longest Common Subsequence (LCS), Coin Change, Fibonacci (DP)
11	0/1 Knapsack, General Coin Change Problem	Correctly solves these optimally

Dynamic Programming Vs Recursion

Feature	Dynamic Programming (DP)	Recursion
Core idea	Recursion + memoization /tabulation	Function calling itself
Overlapping subproblems	Stored & reused	Recomputed again and again
Time complexity	Efficient	Often exponential
Space usage	Table + stack (or only table)	Stack only
Optimization	Highly optimized	No optimization by default
Approach types	Top-down /Bottom-up	Top-down only
Stack overflow risk	Low (tabulation)	High

Development of Dynamic Programming Algorithm

It can be broken into four steps:

→ Characterize the structure of an optimal solution.

→ Recursively define the value of the optimal solution. Like divide and conquer. , divide the problem into two or more optimal parts recursively, This helps to determine what the solution will look like.

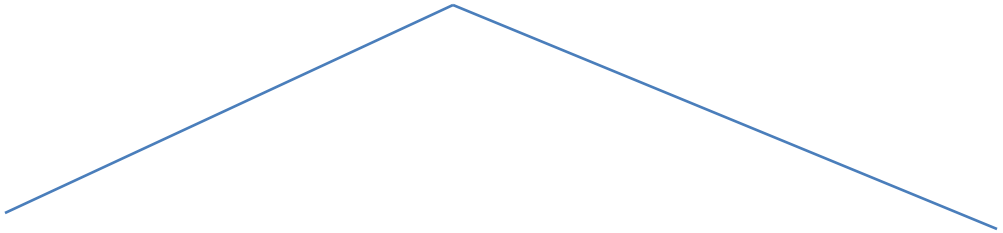
→ Compute the value of the optimal solution from the bottom up (starting with the smallest sub-problems.)

→ Construct the optimal solution for the entire problem from the computed values of the smaller sub-problems.

Matrix Chain Multiplication

A1 . A2 . A3

2×3 3×4 4×2



(A1 . A2) . A3

2×3 3×4 4×2

2×3×4 = 24 0

Total cost = 0+ 24 + 2×4×2 = 40

A1 . (A2 . A3)

2×3 3×4 4×2

0 3×4×2 = 24

Total cost = 0 + 24 + 2×3×2 = 36

→ It is a method under dynamic programming in which previous output is taken as input for next.

→ Here, chain means one matrix's column is equal to the second matrix's row.

→ The algorithm does not perform the multiplications; it just determines the best order in which to perform the multiplications.

→ The order in which we parenthesize the product affects the number of simple arithmetic operations needed to compute the product, or the efficiency.

→ For example, suppose A is a 10 × 30 matrix, B is a 30 × 5 matrix, and C is a 5×60 matrix. Then,

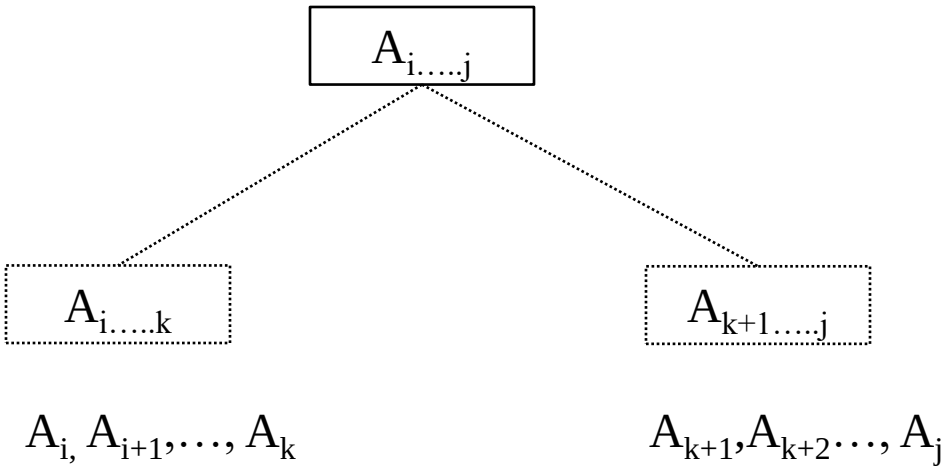
(AB)C = (10 × 30 × 5) + (10 × 5 × 60) = 1500+3000 = 4500 operations.

A(BC) = (30 × 5 × 60) + (10 × 30 × 60)= 9000+18000 = 27000 operations.

→ Even for this small example, considerable savings can be achieved by reordering the evaluation sequence.

→ Given a sequence of matrices A_1, A_2, A_3,….A_n, and dimensions P_0,P_1,P_2,…..P_n, where A_i is the matrix of dimension $P_{i-1} \times P_i$., determine the order of multiplication that minimizes the number of operations.

→ Let $A_{i….j}$ denote the result of multiplying matrices i through j. It is easy to see that $A_{i…j}$ is a $P_{i-1} \times P_j$ matrix. So, for some k, total cost is sum of cost of computing $A_{i….k}$, cost of computing $A_{k+1….j}$, and cost of multiplying $A_{i….k}$ and $A_{k+1….j}$



→ Here, check all the possible sequences of matrices for all possible choices of *k* and take best sequence among them.

Recursive Definition of Optimal solution:

→ Let *m*[*i*, *j*] denote minimum number of scalar multiplications needed to compute $A_{i\ldots j}$

$$m[i,j] = \begin{cases} 0 & if \ \ i = j \\ \min_{i \le k < j} \{m[i,k] + m[k+1,j] + P_{i-1}P_kP_j\} & if \ \ i < j \end{cases}$$

Example 1

Q1. Consider matrices A1, A2, A3 and A4 of order 3×4, 4×5, 5×2, and 2×3. Then find the optimal sequence for the computation of multiplication operation.

Solution:

Given matrices and their order are outlined below:

A1	.	A2	.	A3	.	A4
3×4		4×5		5×2		2×3
P0×P1		P1×P2		P2×P3		P3×P4

We know, for $i = j$, $m[i, j] = 0$

$\therefore m[1,1] = m[2, 2] = m[3, 3] = m[4, 4] = 0$

Now,

$$m[i, j] = \min_{i \le k < j}\{m[i,k] + m[k+1, j] + P_{i-1}P_kP_j\}$$

$$m[1,2] = \min_{1 \le k < 2}\{m[1,k] + m[k+1,2] + P_0P_kP_2\}$$

$$= m[1,1] + m[2,2] + P_0P_1P_2 = 0 + 0 + 3\times4\times5 = 60$$

$$m[i,\ j] = \min_{i \le k < j}\{m[i,k] + m[k+1,\ j] + P_{i-1}P_kP_j\}$$

$$m[2,3] = \min_{2 \le k < 3}\{m[2,k] + m[k+1,3] + P_1P_kP_3\}$$

$$= m[2,2] + m[3,3] + P_1P_2P_3 = 0 + 0 + 4 \times 5 \times 2 = 40$$

$$m[3,4] = \min_{3 \le k < 4}\{m[3,k] + m[k+1,4] + P_2P_kP_4\}$$

$$= m[3,3] + m[4,4] + P_2P_3P_4 = 0 + 0 + 5 \times 2 \times 3 = 30$$

$$m[1,3] = \min_{1 \le k < 3}\{m[1,k] + m[k+1,3] + P_0P_kP_3\}$$

$$= \min\begin{cases} m[1,1] + m[2,3] + P_0P_1P_3 = 0 + 40 + 3 \times 4 \times 2 = 64 \\ m[1,2] + m[3,3] + P_0P_2P_3 = 60 + 0 + 3 \times 5 \times 2 = 90 \end{cases}$$

$$= 64$$

$$m[2,4] = \min_{2 \le k < 4}\{m[2,k] + m[k+1,4] + P_1P_kP_4\}$$

$$= \min\begin{cases} m[2,2] + m[3,4] + P_1P_2P_4 = 0 + 30 + 4 \times 5 \times 3 = 90 \\ m[2,3] + m[4,4] + P_1P_3P_4 = 40 + 0 + 4 \times 2 \times 3 = 64 \end{cases}$$

$$= 64$$

$$m[i,j] = \min_{i \le k < j}\{m[i,k] + m[k+1,j] + P_{i-1}P_kP_j\}$$

$$m[1,4] = \min_{1 \le k < 4}\{m[1,k] + m[k+1,4] + P_0P_kP_4\}$$

$$= \min\begin{cases} m[1,1] + m[2,4] + P_0P_1P_4 = 0 + 64 + 3\times4\times3 = 100 \\ m[1,2] + m[3,4] + P_0P_2P_4 = 60 + 30 + 3\times5\times3 = 135 \\ m[1,3] + m[4,4] + P_0P_3P_4 = 64 + 0 + 3\times2\times3 = 82 \end{cases}$$

$$= 82$$

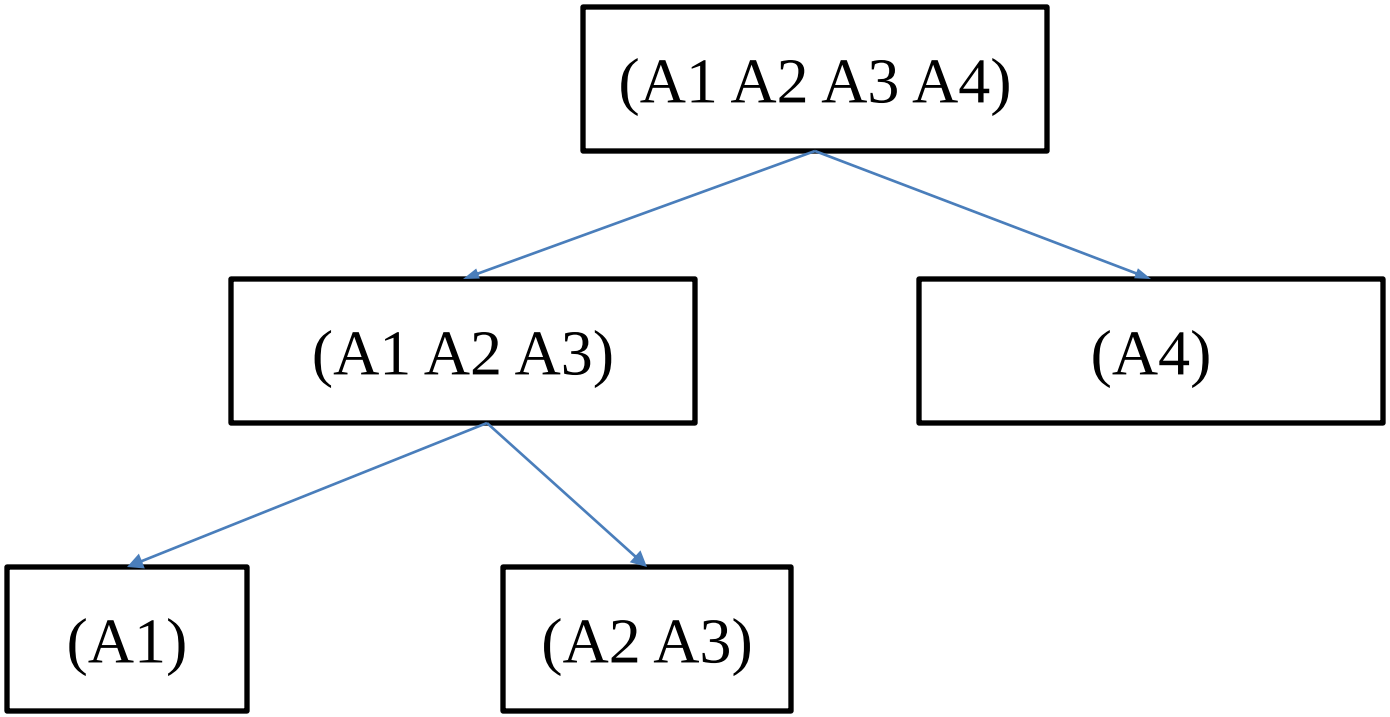
M Table (Cost of Multiplications)

i↓ / j→	**1**	**2**	**3**	**4**
1	0	60	64	**82**
2		0	40	64
3			0	30
4				0

S Table (Point of Parenthesis)

i↓ / j→	**1**	**2**	**3**	**4**
1		1	1	**3**
2			2	3
3				3
4				

So, the optimal multiplication cost = 82.
The optimal multiplication sequence is:



(A1A2A3A4) ⟹ ((A1A2A3)(A4)) ⟹ (((A1)(A2A3))(A4))

This means at first multiply matrix A2 and A3 then multiply their result with matrix A1and finally multiply their result with matrix A4.

Example 2

Consider matrices A1, A2, A3 and A4 of order 3×2, 2×4, 4×2, and 2×5. Then find the optimal sequence for the computation of multiplication operation.

Solution:

Given matrices and their order are outlined below:

A1	.	A2	.	A3	.	A4
3×2		2×4		4×2		2×5
P0×P1		P1×P2		P2×P3		P3×P4

Proceed in the similar way as in example 1 to get the following result:

M Table (Cost of Multiplications)

i↓ / j→	**1**	**2**	**3**	**4**
1	0	24	28	**58**
2		0	16	36
3			0	40
4				0

S Table (Point of Parenthesis)

i↓ / j→	**1**	**2**	**3**	**4**
1		1	1	**3**
2			2	3
3				3
4				

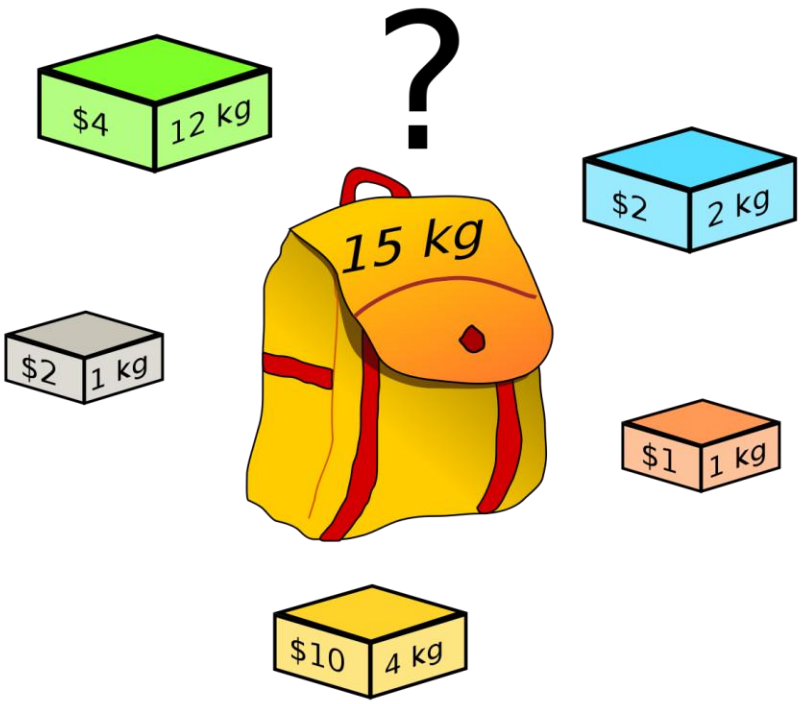
Algorithm

```
Algorithm Matrix-Chain-Multiplication(p)
{
    n=length[p]
    for(i=1;i<n;i++)
            m[i,i]=0
    for(l=2;l<n;l++)
    {
        for(i=1;i<=n-l+1;i++)
        {
            j=i+l-1;
            m[i,j]=infinity
            for(k=i;k<=j-1;k++)
            {
                c=m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j];
                if(c<m[i][j])
                {
                    m[i][j]=c;
                    s[i][j]=k
                }
            }
        }
    }
    return m and s
}
```

Analysis:

The above algorithm can be easily analyzed for running time as $O(n^3)$, due to the nested loops. The space complexity is $O(n^2)$

Zero-one(0/1) Knapsack Problem





→ A thief has a bag or knapsack that can contain maximum weight W of his loot.
→ There are n items and the weight of ith item is wi and its worth is vi.
→ An amount of item can be put into the bag is 0 or 1. i.e. xi is 0 or 1. Here the objective is to collect the items that maximize the total profit earned.
→ Let *W* be the capacity of the knapsack.
→ Let $w = \{w1, w2, w3, \ldots, wn\}$ be weights of items
→ Let $v = \{v1, v2, v3, \ldots, vn\}$ be values of items
→ C[*i*,*w*] = maximum profit earned with item *i* and with the knapsack of capacity w.
→ The recurrence relation for 0/1 knapsack problem is given as:

$$C[i,w] = \begin{bmatrix} 0 & if\ i = 0\ or\ w = 0 \\ C[i-1,w] & if\ wi > w \\ \max\{vi + C[i-1, w-wi], C[i-1,w]\} & if\ i > 0\ and\ wi \le w \end{bmatrix}$$

Algorithm

```
Algorithm DynaKnapsack(W,n,v,w)
{
    for(w=0;w<=W;w++)
        C[0,w]=0;
    for(i=1;i<=n;i++)
        C[i,0]=0;
    for(i=1;i<=n;i++)
    {
        for(w=1;w<=W;w++)
        {
            if(w[i]<=w)
            {
                if v[i]+C[i-1,w-w[i]]>C[i-1,w]
                    C[i,w]= v[i]+C[i-1,w-w[i]]
                else
                    C[i,w]=C[i-1,w];
            }
            else
                C[i,w]=C[i-1,w];
        }
    }
}
```

Analysis:

For run time analysis, examining the above algorithm, the overall run time of the algorithm is $O(nW)$.

Example 1

Let there are four items of weight and values as listed below

W=4

Items = {I1,I2,I3,I4}

w = {1,2,3,2}

v = {2,3,4,1}

Where W be the capacity of knapsack, then find maximum profit earned by using 0/1 knapsack problem.

Solution:

Since there are 4 items and knapsack capacity W = 4. So we need to construct a table of size 5 × 5 as below:

vi	wi	i ↓/w→	0	1	2	3	4
		0	0	0	0	0	0
2	1	1	0	2	2	2	2
3	2	2	0	2	3	5	5
4	3	3	0	2	3	5	6
1	2	4	0	2	3	5	6

Since in the first row, $i = 0$ and in the first column $w = 0$, so $C[i, w] = 0$

For C[1,1]

i =1, w = 1, wi = w1 =1

Since wi ≤ w

So,

C[i, w] = max {vi + C[i-1,w-wi], C[i-1,w]}

∴ C[1, 1] = max {v1 + C[0,0], C[0,1]}

= max{2+0, 0}

=2

For C[1,2]

i =1, w = 2, wi = w1 =1

Since wi ≤ w

So,

C[i, w] = max {vi + C[i-1,w-wi], C[i-1,w]}

∴ C[1, 2] = max {v1 + C[0,1], C[0,2]}

= max{2+0, 0}

=2

For C[1,3]

i =1, w = 3, wi = w1 =1

Since wi ≤ w

So,

C[i, w] = max {vi + C[i-1,w-wi], C[i-1,w]}

∴ C[1, 3] = max {v1 + C[0,2], C[0,3]}

= max{2+0, 0}

=2

For C[1,4]

i =1, w = 4, wi = w1 =1

Since wi ≤ w

So,

C[i, w] = max {vi + C[i-1,w-wi], C[i-1,w]}

∴ C[1, 4] = max {v1 + C[0,3], C[0,4]}

= max{2+0, 0}

=2

For C[2,1]

i =2, w = 1, wi = w2 =2

Since wi ≥ w

So,

C[i, w] = C[i-1,w]

∴ C[2, 1] = C[1, 1]

=2

For C[2,2]

i =2, w = 2, wi = w2 =2

Since wi ≤ w

So,

C[i, w] = max {vi + C[i-1,w-wi], C[i-1,w]}

∴ C[2, 1] = max {v2 + C[1,0], C[1,2]}

= max {3 + 0, 2}

= 3

For C[2,3]

i =2, w = 3, wi = w2 =2

Since wi ≤ w

So,

C[i, w] = max {vi + C[i-1,w-wi], C[i-1,w]}

∴ C[2, 3] = max {v2 + C[1,1], C[1,3]}

= max {3 + 2, 2}

= 5

For C[2,4]

i =2, w = 4, wi = w2 =2

Since wi ≤ w

So,

C[i, w] = max {vi + C[i-1,w-wi], C[i-1,w]}

∴ C[2, 4] = max {v2 + C[1,1], C[1,3]}

= max {3 + 2, 2}

= 5

Similarly,

C[3,1] = 2, C[3,2] = 3, C[3,3]=5, C[3,4] = 6, C[4,1]=2, C[4,2]=3, C[4,3]=5

C[4,4] = 6.

vi	wi	i ↓/W→	0	1	2	3	4
		0	0	0	0	0	0
2	1	1	0	2	2	2	2
3	2	2	0	2	3	5	5
4	3	3	0	2	3	5	6
1	2	4	0	2	3	5	6

Items = {I1,I2,I3,I4}
wi = {1,2,3,2}
vi = {2,3,4,1}

The presence of items in the knapsack is:

x1	**x2**	**x3**	**x4**
1	0	1	0
w1	**w2**	**w3**	**w4**
1	2	3	2

Example 2

Let there are four items of weight and values as listed below

W = 8, n = 4

Items = {I1,I2,I3,I4}

w = {3,4,6,5}

v = {2,3,4,1}

Where W be the capacity of knapsack, then find maximum profit earned by using 0/1 knapsack problem.

Solution:

Since there are 4 items and knapsack capacity W = 8. So we need to construct a table of size 5 × 9 as below:

		i ↓/w→	0	1	2	3	4	5	6	7	8
vi	wi	0	0	0	0	0	0	0	0	0	0
2	3	1	0	0	0	2	2	2	2	2	2
3	4	2	0	0	0	2	3	3	3	5	5
4	5	3	0	0	0	2	3	4	4	5	6
1	6	4	0	0	0	2	3	4	4	5	6

Items of weight 5 and 3 are selected.

Thank You!

Any Queries?

Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms

Chapter 5. Dynamic Programming

Session 2

by:

Er. Bikal Adhikari

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

Chapter Outline

Chapter 5. Dynamic Programming

8 hours

5.1 Basic concepts

5.2 All pair shortest path algorithm

5.3 Travelling salesperson problem

5.4 String editing

5.5 0/1 knapsack problem using dynamic programming

5.6 Matrix chain multiplication

5.7 Flow shop scheduling

Floyd Warshall Algorithm

→ The algorithm being discussed uses dynamic programming approach.

→ The algorithm being presented works even if some of the edges have negative weights.

→ It is mainly used to find all pair shortest path of the given weighted graph.

→ Consider a weighted graph $G = (V, E)$ and denote the weight of edge connecting vertices i and j by w_{ij}.

→ Let W be adjacency matrix for the given graph G.

→ Let D^k denote an $n \times n$ matrix such that $D^k(i, j)$ is defined as the weight of shortest path from vertex i to vertex j using only vertices from 1, 2,…, k as intermediate vertices in the path.

→ If we consider shortest path with intermediate vertices as above, then computing the path contains two cases.

→ First case: $D^k(i, j)$ does not contain k as intermediate vertex

→ Second case: $D^k(i, j)$ contains k as intermediate vertex.

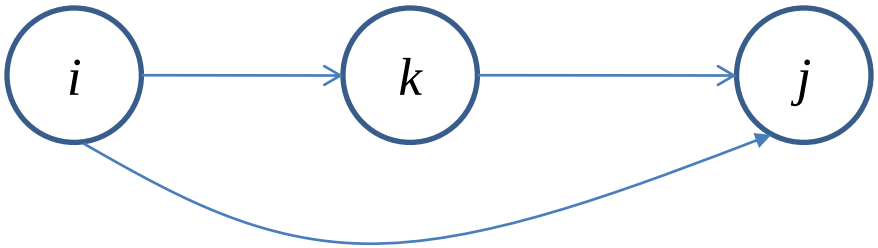
→ We have following relations:

$$D^k(i,j) = \begin{cases} D^{k-1}(i,j) & \text{, when k is not an intermediate vertex} \\ D^{k-1}(i,k) + D^{k-1}(k,j) & \text{, when k is an intermediate vertex} \end{cases}$$

→ So from above relations we obtain:

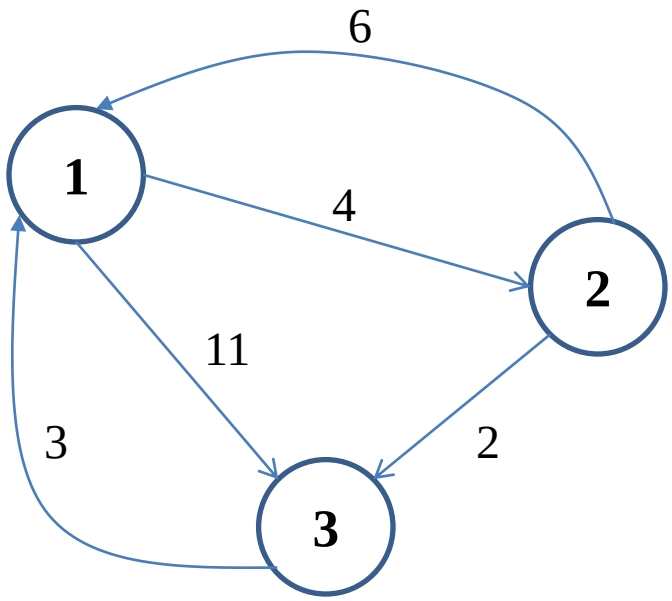
$$D^k(i,j) = \min\{D^{k-1}(i,j), D^{k-1}(i,k) + D^{k-1}(k,j)\}$$

→ The above relation is used by Floyd's algorithm to compute all pairs shortest path in bottom up manner for finding D^1, D^2, D^3, ….D^n.



Example

Q. Find the shortest path from every vertex to other vertices in the following graph:



Solution:

Adjacency matrix of given graph is:

W or D^0	**1**	**2**	**3**
1	0	4	11
2	6	0	2
3	3	∞	0

Example

Case 1: When vertex (1) as an intermediate vertex

$D^1(1,1) = 0$

$D^1(1,2) = \min\{D^0(1,2), D^0(1,1) + D^0(1,2)\} = unchanged = \min\{4, 0+4\} = 4$

$D^1(1,3) = \min\{D^0(1,3), D^0(1,1) + D^0(1,3)\} = unchanged = \min\{11, 0+11\} = 11$

$D^1(2,1) = \min\{D^0(2,1), D^0(2,1) + D^0(1,1)\} = unchanged = \min\{6, 6+0\} = 6$

$D^1(2,2) = 0$

$D^1(2,3) = \min\{D^0(2,3), D^0(2,1) + D^0(1,3)\} = may\ change = \min\{2, 6+11\} = 2$

$D^1(3,1) = \min\{D^0(3,1), D^0(3,1) + D^0(1,1)\} = unchanged = \min\{3, 3+0\} = 3$

$D^1(3,2) = \min\{D^0(3,2), D^0(3,1) + D^0(1,2)\} = may change = \min\{\infty, 3+4\} = 7$

$D^1(3,3) = 0$

The adjacency matrix can be modified as:

D^1	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0

Case 2: When vertex (2) as an intermediate vertex

$$D^2(1,1) = 0$$

$$D^2(1,2) = unchanged = 4$$

$$D^2(1,3) = \min\{D^1(1,3), D^1(1,2) + D^1(2,3)\} = may\,change = \min\{11, 4+2\} = 6$$

$$D^2(2,1) = unchanged = 6$$

$$D^2(2,2) = 0$$

$$D^2(2,3) = unchanged = 2$$

$$D^2(3,1) = \min\{D^1(3,1), D^1(3,2) + D^1(2,1)\} = may\,change = \min\{3, 7+6\} = 3$$

$$D^2(3,2) = unchanged = 7$$

$$D^2(3,3) = 0$$

The adjacency matrix can be modified as:

D^2	1	2	3
1	0	4	6
2	6	0	2
3	3	7	0

Case 3: When vertex (3) as an intermediate vertex

$$D^3(1,1) = 0$$

$$D^3(1,2) = \min\{D^2(1,2), D^2(1,3) + D^2(3,2)\} = may\,change = \min\{4, 6+7\} = 4$$

$$D^3(1,3) = unchanged = 6$$

$$D^3(2,1) = \min\{D^2(2,1), D^2(2,3) + D^2(3,1)\} = may\,change = \min\{6, 2+3\} = 5$$

$$D^3(2,2) = 0$$

$$D^3(2,3) = unchanged = 2$$

$$D^3(3,1) = unchanged = 3$$

$$D^3(3,2) = unchanged = 7$$

$$D^3(3,3) = 0$$

The adjacency matrix can be modified as:

D^3	**1**	**2**	**3**
1	0	4	6
2	5	0	2
3	3	7	0

D^3	1	2	3
1	0	4	6
2	5	0	2
3	3	7	0

Thus,
The shortest path from vertex 1 to 1 = 0
The shortest path from vertex 1 to 2 = 4
The shortest path from vertex 1 to 3 = 6
The shortest path from vertex 2 to 1 = 5
The shortest path from vertex 2 to 2 = 0
The shortest path from vertex 2 to 3 = 2
The shortest path from vertex 3 to 1 = 3
The shortest path from vertex 3 to 2 = 7
The shortest path from vertex 3 to 3 = 0

```
Algorithm FloydWarsal_ASP(W,D,n)
{
for(i=1;i<=n;i++)
{
    for(j=1;j<=n;j++)
    {
    D[i][j]=W[i][j]; //Original D0 matrix
    }
}
for(k=1;k<=n;k++)
{
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            if(D[i][j]> D[i][k]+D[k][j]) then
                Set, D[i][j]=D[i][k]+D[k][j]
        }
    }
}
}
```

Analysis:

The running time of the Floyd Warshall algorithm is determined by the triply nested for loops. Each loop executes at most O(n) times. The algorithm thus runs in time $O(n^3)$

Travelling Salesman Problem

- → In travelling salesman problem, a salesman must visit *n* cities.
- → We can say that salesman wishes to make a tour or Hamiltonian cycle, visiting each city exactly once and finishing at the city he started from.
- → There is no negative cost $C(i, j)$ to travel from city *i* to city *j*.
- → The goal is to find a tour of minimum cost.
- → We assume that every two cities are connected. Such problems are called Travelling Salesman problem(TSP).
- → The travelling salesman problem can be solved either by dynamic programming paradigm *or* non-dynamic programming ways.
- → In this topic, we discuss dynamic implementation of TSP.

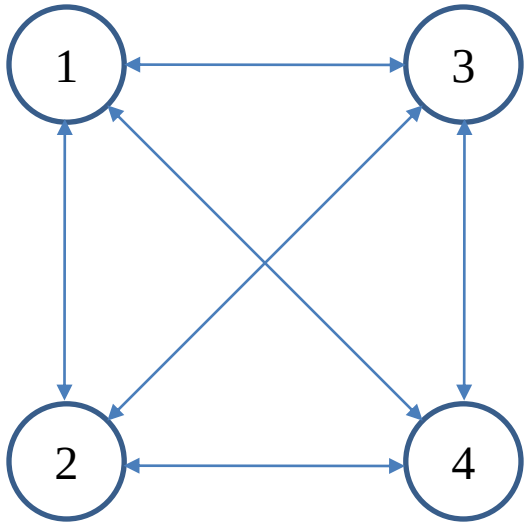
→ Let *i* be the starting vertex, *S* be the set of remaining vertices except vertex *i*, *k* be any one vertex of *S* and *g*(*i*,*S*) be the minimum travel cost of *TSP*.

→ Then their recurrence relation can be defined as below:

$$g(i,S) = \min_{k \in S} \{C_{ik} + g(k, S - \{k\}\}$$

Example

Q. For the following graph find minimum cost tour for the traveling salesman problem



Cost adjacency matrix =

	1	**2**	**3**	**4**
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Solution:

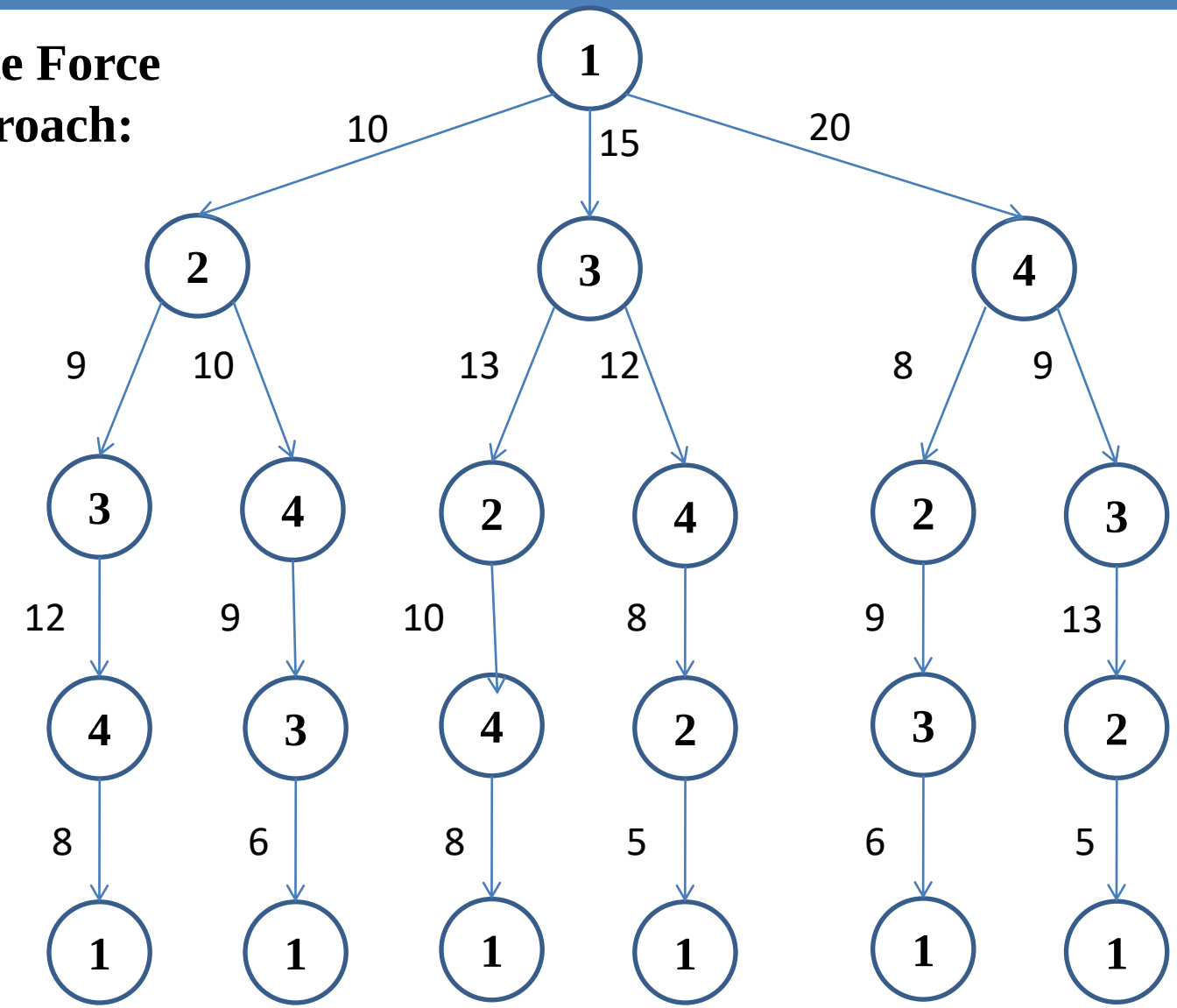
Greedy Approach:

Suppose we are in node 1 at first. By using greedy approach following path is followed:

Path: 1 → 2 → 3 → 4 → 1

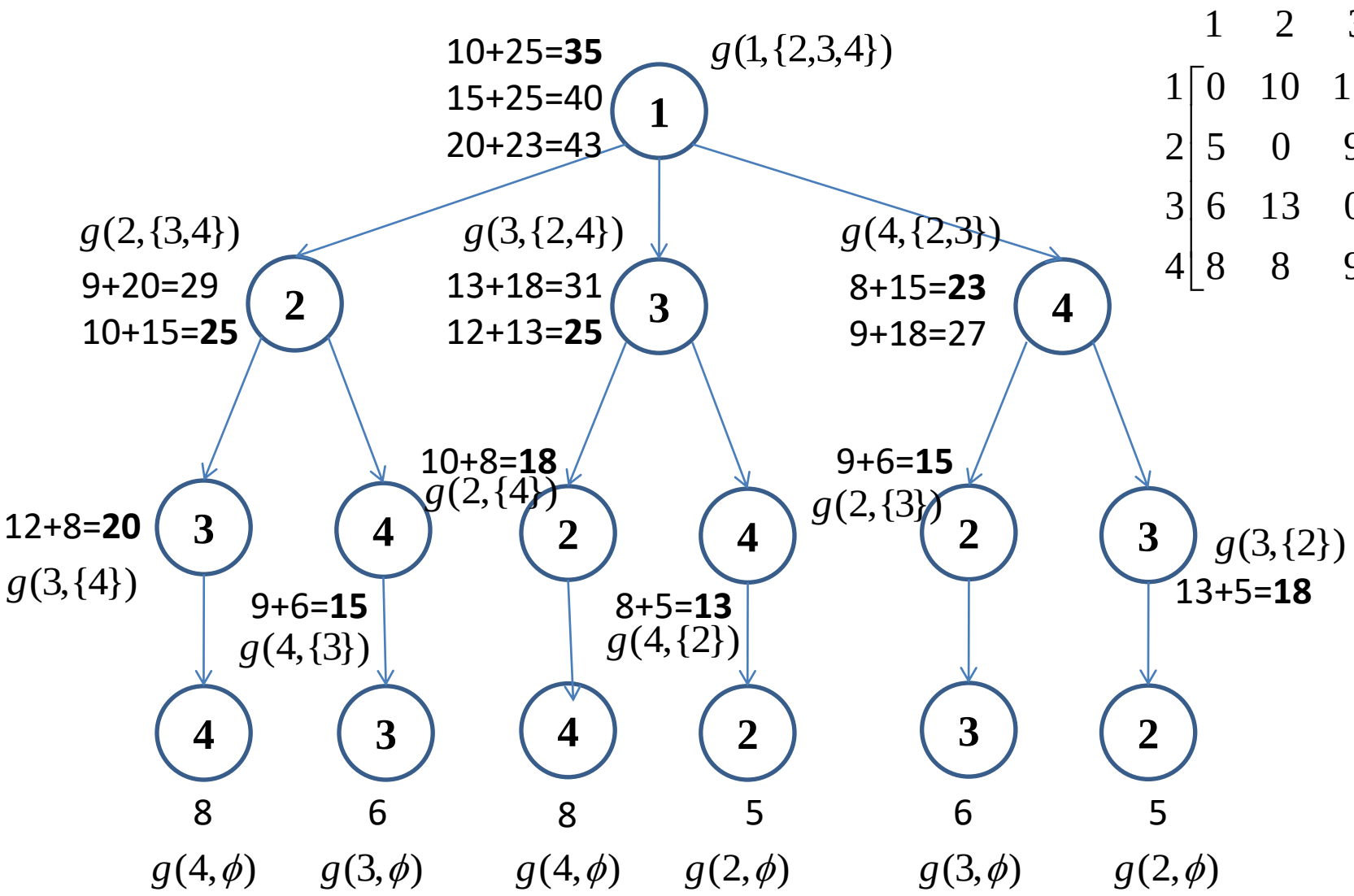
Length: 10+9+12+8 = 39

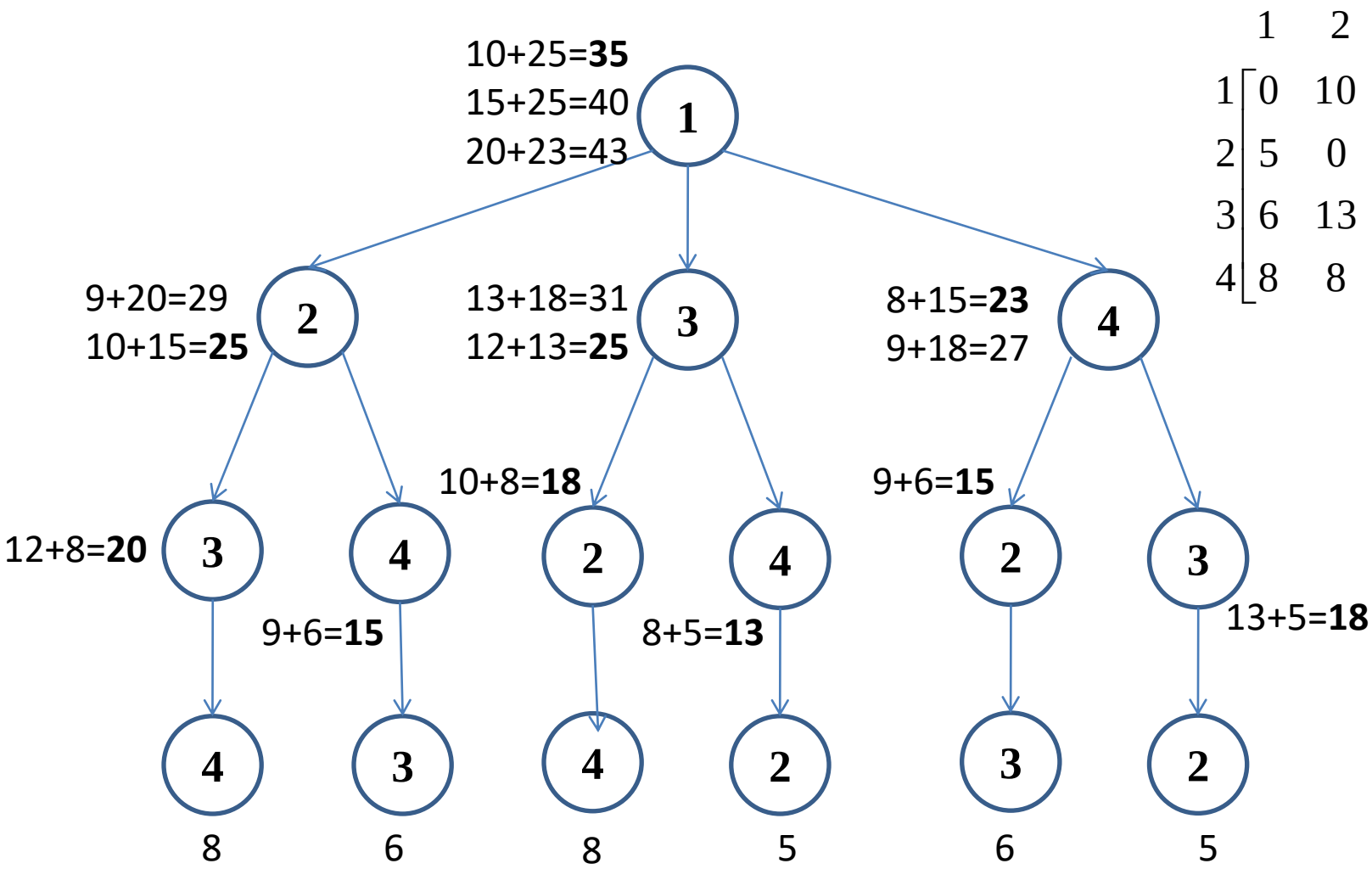
Brute Force Approach:



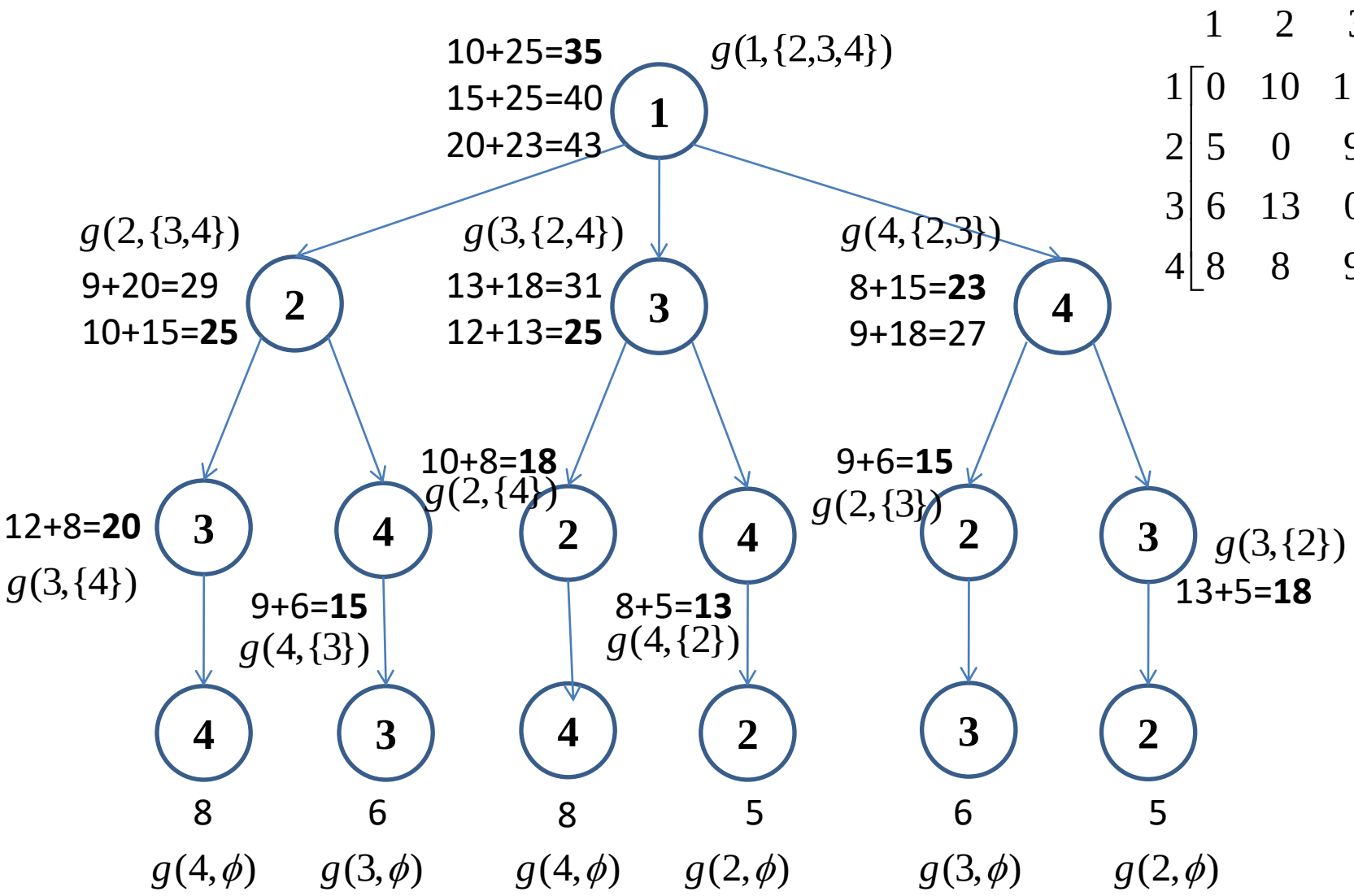
	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Total Cost: 39 35 46 40 43 47


1 2 3 4
1 0 10 15 20
2 5 0 9 10
3 6 13 0 12
4 8 8 9 0
10+25=35
15+25=40
20+23=43
g(1,{2,3,4})
1
g(2,{3,4})
9+20=29
10+15=25
2
g(3,{2,4})
13+18=31
12+13=25
3
g(4,{2,3})
8+15=23
9+18=27
4
12+8=20
g(3,{4})
3
4
10+8=18
g(2,{4})
2
4
9+6=15
g(2,{3})
2
3
g(3,{2})
13+5=18
9+6=15
g(4,{3})
8+5=13
g(4,{2})
4
3
4
2
3
2
8
6
8
5
6
5
g(4,φ)
g(3,φ)
g(4,φ)
g(2,φ)
g(3,φ)
g(2,φ)

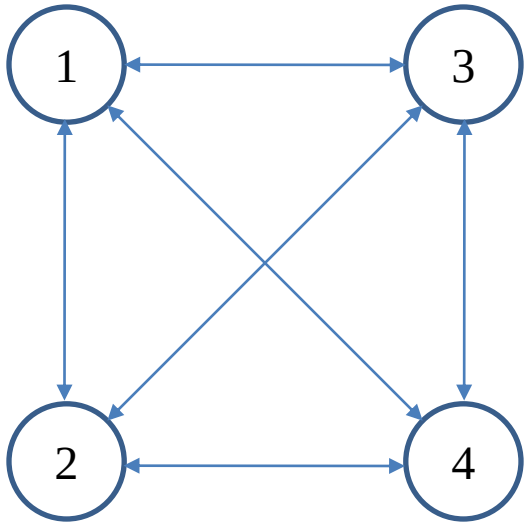


	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0


1 2 3 4
1 0 10 15 20
2 5 0 9 10
3 6 13 0 12
4 8 8 9 0
10+25=35
15+25=40
20+23=43
g(1,{2,3,4})
1
g(2,{3,4})
9+20=29
10+15=25
2
g(3,{2,4})
13+18=31
12+13=25
3
g(4,{2,3})
8+15=23
9+18=27
4
12+8=20
g(3,{4})
3
4
10+8=18
g(2,{4})
2
4
9+6=15
g(2,{3})
2
3
g(3,{2})
13+5=18
9+6=15
g(4,{3})
8+5=13
g(4,{2})
4
3
4
2
3
2
8
6
8
5
6
5
g(4,φ)
g(3,φ)
g(4,φ)
g(2,φ)
g(3,φ)
g(2,φ)

Example

Q. For the following graph find minimum cost tour for the traveling salesman problem



Cost adjacency matrix =

	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Solution: Let S= {1, 2, 3, 4}

Let us start the tour from vertex 1:

$$g(1, S-\{1\}) = \min\{C_{1k} + g(k, S-\{1,k\})\}$$

$$2 < k < n$$

More generally writing, $g(i,S) = \min\{C_{ik} + g(k, S-\{k\})\}$ ……..(ii)

We know, $g(i,\phi) = C_{i1}, 1 \le i \le n$ *so,*

$g(2,\phi) = C_{21} = 5$

$g(3,\phi) = C_{31} = 6$

$g(4,\phi) = C_{41} = 8$

Using equation (ii), we obtain,

g(2,{3})=min{C23+g(3, Φ)}=9+6=15

g(2,{4})=min{C24+g(4, Φ)}=10+8=18

g(3,{2})=min{C32+g(2, Φ)}=13+5=18

g(3,{4})=min{C34+g(4, Φ)}=12+8=20

g(4,{2})=min{C42+g(2, Φ)}=8+5=13

g(4,{3})=min{C43+g(3, Φ)}=9+6=15

g(2,{3,4})=min{C23+g(3,{4}), C24+g(4,3)}=min{9+20,10+15}=25

g(3,{2,4})=min{C32+g(2,{4}), C34+g(4,{2})}=min{13+18,12+13}=25

g(4,{2,3})=min{C42+g(2,{3}), C43+g(3,{2})}=min{8+15,9+18}=23

g(1,{2,3,4})=min{C12 + g(2,{3,4}), C13+g(3,{2,4}),C14+g(4,{2,3})}

= min{10+25,15+25,20+23} = 35

Hence, the optimal tour for the graph has length = 35
The optimal tour is 1,2,4,3,1.

Time Complexity

$$O(n^2.2^n)$$

Thank You!

Any Queries?

Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms
Chapter 5. Dynamic Programming
Session 3

by:

Er. Bikal Adhikari

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

Chapter Outline

Chapter 5. Dynamic Programming

8 hours

5.1 Basic concepts

5.2 All pair shortest path algorithm

5.3 Travelling salesperson problem

5.4 String editing

5.5 0/1 knapsack problem using dynamic programming

5.6 Matrix chain multiplication

5.7 Flow shop scheduling

Longest Common Subsequence(LCS) Problem

→ This method is used to test the matching or similarities between the two strings.

→ A subsequence of a given sequence is just the given sequence with some elements left out.

→ Given two sequences X and Y, we say that the sequence Z is a common of sequence X and Y if Z is a subsequence of both X and Y.

→ In the longest common subsequence problem, we are given two sequences $X = (x1\ x2 \ldots xm)$ and $Y = (y1\ y2 \ldots yn)$ and wish to find a maximum length common subsequence of X and Y.

→ LCS problem can be solved using dynamic programming.

→ For example: Let X=(ABRACADABRA) and Y=(YABBADABBAD). Then, the longest common subsequence is Z = (ABADABA).

Recurrence Relation for LCS:

→ Let , x_i and y_j represent any two sequences of characters.

→ Let, $C[i, j]$ represents the LCS of x_i and y_j.

→ Then, it's recurrence relation is:

$$C[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i-1, j-1] + 1 & \text{if } x_i = y_j \\ \max\{C[i-1, j], C[i, j-1]\} & \text{if } i > 0, j > 0 \; x_i \neq y_j \end{cases}$$

Algorithm:

```
Algorithm LCS(X,Y)
{
m=length(X);
n=length(Y);
for(i=1;i<=m;i++)
    c[i,0]=0
for(j=0;j<=n;j++)
    c[0,j]=0;
for(i=1;i<=m;i++)
{
    for(j=1;j<=n;j++)
    {
        if(X[i]==Y[j])
            c[i][j]=c[i-1][j-1]+1; b[i][j]="upleft";
        else if(c[i-1][j]>=c[i][j-1])
            c[i][j]=c[i-1][j]; b[i][j]="up";
        else
            c[i][j]=c[i][j-1];b[i][j]="left";
    }
    return b and c;
}
}
```

Analysis:

This algorithm can be easily analyzed for running time as $O(mn)$, due to two nested loops. The space complexity is $O(mn)$.

Examples

Example 1: Given two sequences X = [ABCBDAB] and Y = [BDCABA]. Find the longest common subsequence to both.

Solution:

X = [ABCBDAB] and Y = [BDCABA]

m = length(X) = 7 and n= length(Y)=6

Here,

$x1$ = X[1]=A

x2=B

x3=C

x4=B

x5=D

x6=A

x7=B

Here,

$y1$ = Y[1]=B

y2=D

y3=C

y4=A

y5=B

y6=A

For i = 1 to 7.
$C[i, 0] = 0$

For j=1 to 6.
$C[0, j] = 0$

***i*↓**	***j*→**	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0						
2	B	0						
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑					
2	B	0						
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑				
2	B	0						
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑			
2	B	0						
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1		
2	B	0						
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	
2	B	0						
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0						
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1					
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1				
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1			
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑		
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓		*j*→ **0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0						
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑					
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑				
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2			
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓		*j*→ **0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2		
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓		*j*→ **0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓		*j*→ **0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0						
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓		*j*→ **0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1					
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑				
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓		*j*→ **0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑			
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑		
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0						
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑					
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2				
6	A	0						
7	B	0						

Now filling the table,

i↓		*j*→ **0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑			
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑		
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	
6	A	0						
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0						
7	B	0						

Now filling the table,

i↓		*j*→ **0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑					
7	B	0						

Now filling the table,

i↓		*j*→ **0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑				
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑	2 ↑			
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑	2 ↑	↖ 3		
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑	2 ↑	↖ 3	3 ↑	
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑	2 ↑	↖ 3	3 ↑	↖ 4
7	B	0						

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑	2 ↑	↖ 3	3 ↑	↖ 4
7	B	0	↖ 1					

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑	2 ↑	↖ 3	3 ↑	↖ 4
7	B	0	↖ 1	2 ↑				

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑	2 ↑	↖ 3	3 ↑	↖ 4
7	B	0	↖ 1	2 ↑	2 ↑			

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑	2 ↑	↖ 3	3 ↑	↖ 4
7	B	0	↖ 1	2 ↑	2 ↑	3 ↑		

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑	2 ↑	↖ 3	3 ↑	↖ 4
7	B	0	↖ 1	2 ↑	2 ↑	3	↖ 4	

Now filling the table,

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑	2 ↑	↖ 3	3 ↑	↖ 4
7	B	0	↖ 1	2 ↑	2 ↑	3	↖ 4	4 ↑

Longest Common Subsequence = BCBA , Length =4

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	B	D	C	A	B	A
0	X	0	0	0	0	0	0	0
1	A	0	0 ↑	0 ↑	0 ↑	↖ 1	← 1	↖ 1
2	B	0	↖ 1	← 1	← 1	1 ↑	↖ 2	← 2
3	C	0	1 ↑	1 ↑	↖ 2	← 2	2 ↑	2 ↑
4	B	0	↖ 1	1 ↑	2 ↑	2 ↑	↖ 3	← 3
5	D	0	1 ↑	↖ 2	2 ↑	2 ↑	3 ↑	3 ↑
6	A	0	1 ↑	2 ↑	2 ↑	↖ 3	3 ↑	↖ 4
7	B	0	↖ 1	2 ↑	2 ↑	3	↖ 4	4 ↑

Formula Verification for cells other than that having *i* or *j* equal to 0:

Calculation of C[2,5]:

Here, x2 =y5

∴ C[i,j] = C[i-1,j-1]+1

∴ C[2,5] = C[1,4]+1= 1+1=2

∴ arrow[2,5]= '↖'

Calculation of C[5,5]:

Here, x5 ≠ y5

∴ C[i, j]=max{C[i-1,j],C[i,j-1]}

∴ C[5,5] = max{C[4,5]+C[5,4]}=max{3,2} = 3

∴ arrow[5,5]= '↑'

Calculation of C[4,6]:

Here, x4 ≠ y6

∴ C[i, j]=max{C[i-1,j],C[i,j-1]}

∴ C[4,6] = max{C[3,6]+C[4,5]}=max{2,3}=3

∴ arrow[4,6]= '←'

Example 2: Find the LCS of X = abbabba and Y = aaabba

***Solution*:** LCS = aabba, length = 5

i↓	*j*→	**0**	**1**	**2**	**3**	**4**	**5**	**6**
		Y	***a***	***a***	***a***	***b***	***b***	***a***
0	X	0	0	0	0	0	0	0
1	***a***	0	↖ 1	↖ 1	↖ 1	← 1	← 1	↖ 1
2	***b***	0	1 ↑	↖ 1 ↑	1 ↑	↖ 2	↖ 2	← 2
3	***b***	0	1 ↑	↖ 1 ↑	1 ↑	↖ 2	↖ 3	← 3
4	***a***	0	↖ 1	↖ 2	↖ 2	2 ↑	3 ↑	↖ 4
5	***b***	0	1 ↑	2 ↑	2 ↑	↖ 3	↖ 3	4 ↑
6	***b***	0	1 ↑	2 ↑	2 ↑	↖ 3	↖ 4	4 ↑
7	***a***	0	↖ 1	↖ 2	↖ 3	3 ↑	4 ↑	↖ 5

Edit Distance

- The minimum edit distance between two strings
- Is the minimum number of editing operations
  - Insertion
  - Deletion
  - Substitution
- Needed to transform one into the other

Minimum Edit Distance

- Two strings and their **alignment**:
- Given two sequences, an alignment is a correspondence between substrings of the two sequences, like the individual letters in this case

```
I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N
```

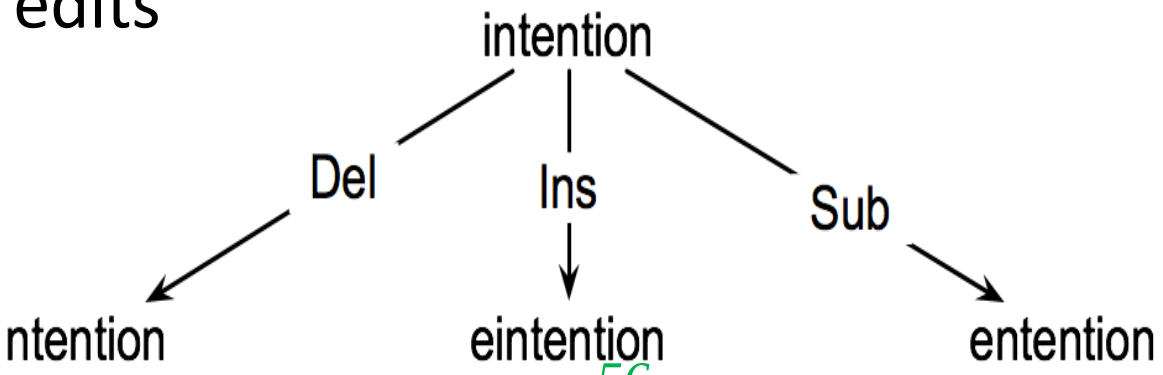
We can read off the edit distance from the alignment

```
I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N
d s s   i s
```

- If each operation has cost of 1
  - Distance between these is 5

How to find the Min Edit Distance?

- Searching for a path (a sequence of edits) from the start string to the final string:
  - **Initial state**: the word we're transforming
  - **Operators**: insert, delete, substitute
  - **Goal state**: the word we're trying to get to
  - **Path cost**: what we want to minimize: the number of edits



Minimum Edit as Search

- But the space of all edit sequences is huge!
  - We can’t afford to navigate naively
- Luckily:
  - Lots of distinct paths wind up at the same state.
  - We don’t have to keep track of all of them
  - Just the shortest path to each of those revisited states.
  - We'll see a dynamic programming solution in the next lecture

Defining Min Edit Distance

- For two strings
  - X of length *n*
  - Y of length *m*
- We define D(*i,j*)
  - the edit distance between X[1..*i*] and Y[1..*j*]
    - i.e., the first *i* characters of X and the first *j* characters of Y
  - The edit distance between X and Y is thus D(*n,m*)

Dynamic Programming for Minimum Edit Distance

- **Dynamic programming**: A tabular computation of D(*n,m*)
- Solving problems by combining solutions to subproblems.
- Bottom-up
  - We compute D(i,j) for small *i,j*
  - And compute larger D(i,j) based on previously computed smaller values
  - i.e., compute D(*i,j*) for all *i* (0 < *i* < n) and *j* (0 < j < m)

Edit Distance

Base conditions:

$D(i,0) = i \;,\; D(0,j) = j$

Termination:

$D(N,M)$ is distance

Recurrence Relation:

For each $i = 1 \ldots M$

  For each $j = 1 \ldots N$

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 & \text{deletion} \\ D(i,j-1) + 1 & \text{insertion} \\ D(i-1,j-1) + \begin{cases} 1; \text{ if } X(i) \neq Y(j) & \text{substitution} \\ 0; \text{ if } X(i) = Y(j) & \end{cases} & \end{cases}$$

$$ptr(i,j) = \begin{cases} LEFT & \text{insertion} \\ DOWN & \text{deletion} \\ DIAG & \text{substitution} \end{cases}$$

Edit Distance



		"" 0	r 1	o 2	s 3
""	0	0	1	2	3
h	1	1	1	2	3
o	2	2	2	1	2
r	3	3	2	2	2
s	4	4	3	3	2
e	5	5	4	4	3

Edit Distance



Memoization Strategy

→ Memoization means recording the results of earlier calculations so that we don't have to repeat the calculation later.

→ If our code depends on the results of earlier calculations, and if the same calculations are performed over and over again, then it makes sense to store interim results so that we can avoid repeating the math.

→ So far we have talked about implementing dynamic programming in a bottom up fashion.

→ Dynamic programming can also be implemented using memoization.

→ With memoization, we implement the algorithm recursively, but we keep track of all of the sub solutions. If we encounter a sub-problem that we have seen, we look up the solution. If we encounter a sub-problem we have not seen, we compute it and add it to the list of sub solutions we have seen. Each subsequent time that the sub problem is encountered, the value stored in the table is simply looked up and returned.

→ Memoization offers the efficiency of dynamic programming. It maintains the top-down recursive strategy.

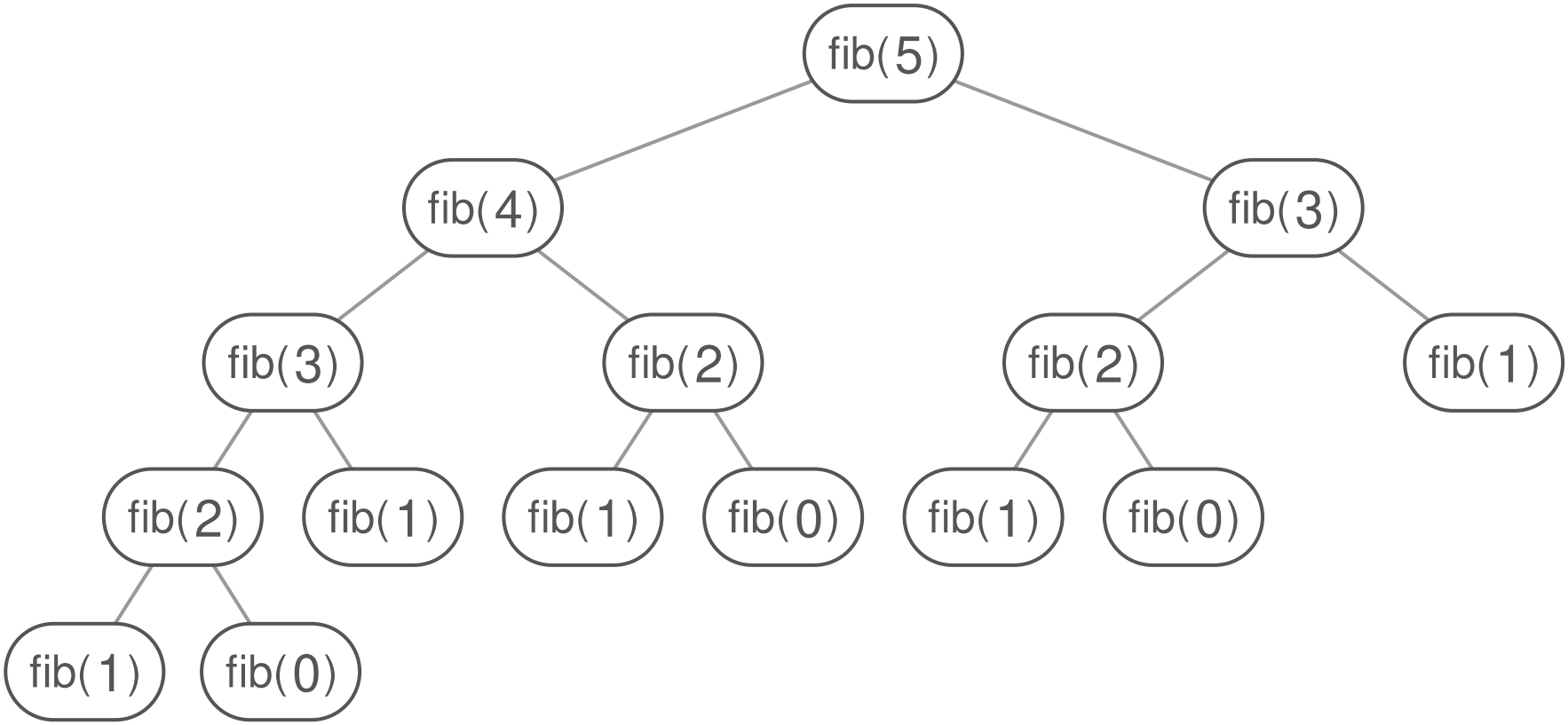
Example:

→ Lets take a problem of finding Fibonacci number by using recursion as below:

```
int fib(int n)
{
if(n==0)
    return 0;
else if(n==1)
    return 1;
else
    fib(n-1)+fib(n-2);
}
```

→ The recurrence relation is:

$$fib(n) = \begin{cases} 0 & if\ n = 0 \\ 1 & if\ n = 1 \\ fib(n-1) + fib(n-2) & if\ n \geq 2 \end{cases}$$



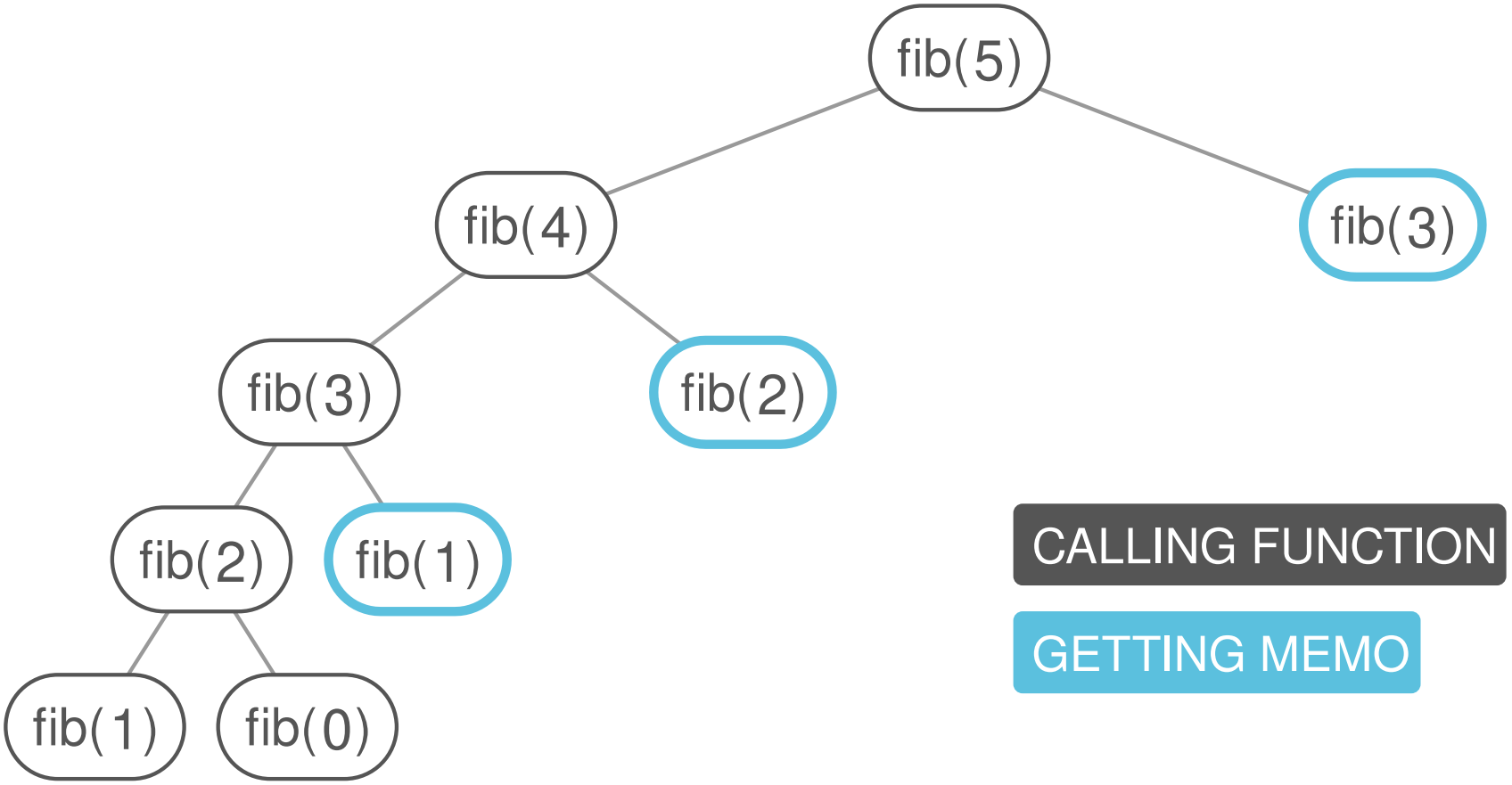
→ Since, the recurrence relation for this problem is:

$$T(n) = 2T(n-1) + 1$$

By solving this, we get,

$$T(n) = O(2^n)$$

→ Now, using memoization:



→ From the above tracing using memoization, there are only 6 calls for n = 5. Hence, for the problem of size n, there are n+1 calls.

→ Thus, total time complexity is $T(n) = n + 1 = O(n)$

Thank You!
Any Queries?

Flowshop Scheduling

→ In a general flow shop we may have, n jobs each having m tasks $T_{1i}, T_{2i}, \ldots T_{mi}$, where $1 \leq i \leq n$

→ Task T_{ji} is to be performed on processor P_j, where $1 \leq j \leq m$

→ The time required to complete task T_{ji} is t_{ji}

→ A schedule for the n jobs is an assignment of tasks to time intervals on the processors.

→ No processor may have more than one task assigned to it in any time interval.

→ For any job i the processing of tasks T_{ji}, $j > 1$, will not be started until task $T_{j\text{-}1,i}$ has been completed.

→ In this problem we are given a set of jobs each having some tasks to be performed. Processors for each of the tasks.

→ The objective is to determine the order of assigning the tasks to processors such that the sum finishing time is minimal.

Notation

$$[J] = \begin{array}{c} \\ P_1 \\ P_2 \\ \vdots \\ \\ P_m \end{array} \begin{array}{c} \begin{array}{cccc} J_1 & J_2 & \cdots\cdots & J_n \end{array} \\ \left\lfloor \begin{array}{cccc} T_{11} & T_{12} & \cdots\cdots & T_{1n} \\ T_{21} & T_{22} & \cdots\cdots & T_{2n} \\ \vdots & \vdots & & \vdots \\ T_{m1} & T_{m2} & \cdots\cdots & T_{mn} \end{array} \right\rfloor_{m \times n} \end{array}$$

Figure: Generic mathematical notation representing ***n*** *Jobs each consisting of* ***m*** *Tasks to be processed in* ***m*** *Processors.*

Example

Two jobs have to be scheduled on three processors. The task times are given in the matrix below:

$$\begin{matrix} & J_1 & J_2 \end{matrix}$$
$$[J] = \begin{bmatrix} 2 & 0 \\ 3 & 3 \\ 5 & 2 \end{bmatrix} = \begin{bmatrix} T_{11} & T_{12} \\ T_{21} & T_{22} \\ T_{31} & T_{32} \end{bmatrix}$$

J_1 consists of tasks T_{11}, T_{21}, and T_{31}

J_2 consists of tasks T_{12}, T_{22}, and T_{32}

In general, T_{ji} means j^{th} task of ith job.

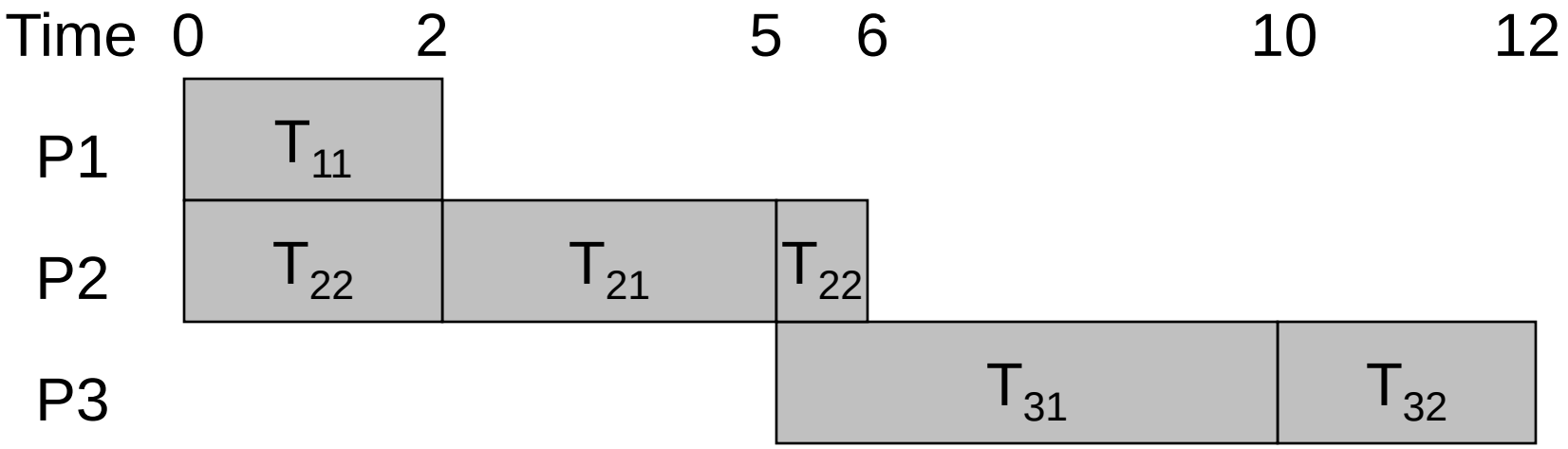


Figure (a)

This is a preemptive schedule.

A *preemptive schedule* is the one which allows the processing of a task to be terminated before its completion.

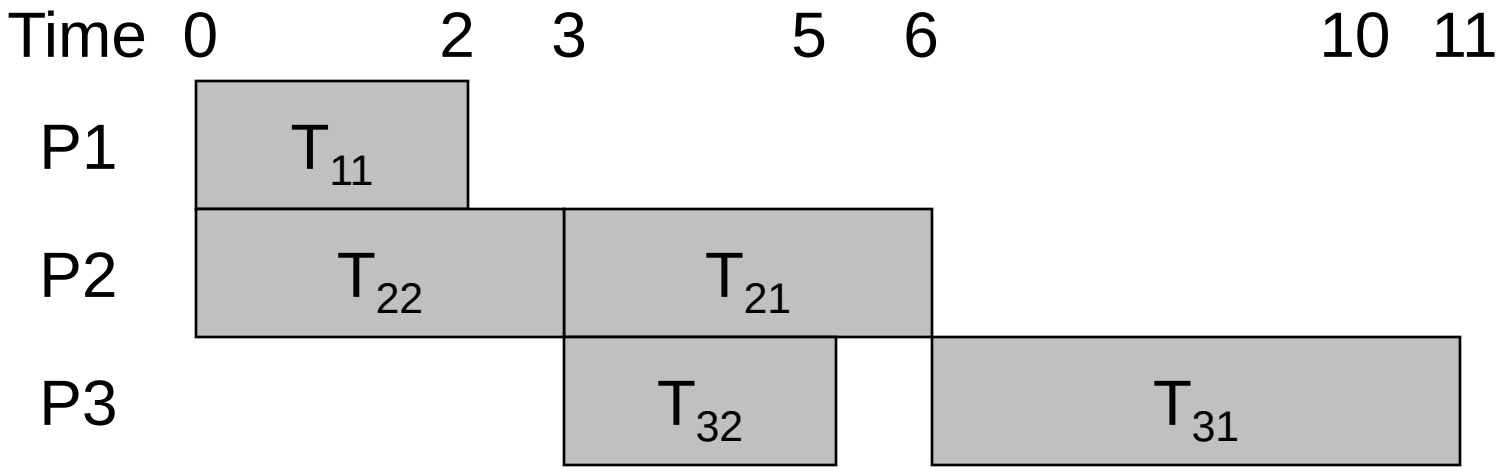


Figure (b)

This is a non-preemptive schedule.

A *non-preemptive schedule* is a schedule in which the processing of a task on any processor is not terminated until the task is complete.

Terminologies

→ The finish time $f_i(S)$ of job i is the time at which all tasks of job i have been completed in schedule S.

→ In figure (a), $f_1(S)=10$ and
$f_2(S)=12$

→ In figure (b), $f_1(S)=11$ and
$f_2(S)=5$

→ The Finish Time $F(S)$ of a schedule S is given by ;

$$F(S)=\max\{f_i(S)\}$$

where, $1 \leq i \leq n$.

→ The Mean Flow Time, MFT(S) is defined to be:

$$MFT(S)=(\Sigma f_i(S))/n$$

where, $1 \leq i \leq n$

Constraint and Assumptions

→ The general problem of obtaining schedules for $m > 2$ is computationally difficult, dynamic programming leads to an efficient algorithm to obtain OFT (optimal finish time) schedules for the case $m = 2$ (tasks=2).

→ $t_{1i} \rightarrow a_i$, where t_{1i} is time of task 1 of job i

→ $t_{2i} \rightarrow b_i$, where t_{2i} is time of task 2 of job i

→ Case being considered,

$a_i \neq 0,\ 1 \leq i \leq n$

→ Special case,

If $a_i = 0$, then

All jobs with $a_i \neq 0$ are optimally scheduled and then all jobs with $a_i = 0$ are simply added to it.

Derivation of DP Formulation

Let σ_1, σ_2, σ_3,…..σ_k be a permutation prefix defining a schedule for jobs J_1, J_2, J_3,…J_k.

$f_1 \rightarrow$ time taken by J_1, J_2,…J_k on processor P_1

$f_2 \rightarrow$ time taken by J_1, J_2,…J_k on processor P_2

Let, $t = f_2 - f_1$

The task, T_{2i} cannot be started until $max\{a_i - t\}$.

Hence, $f_2 - f_1 = b_i + max\{a_i, t\} - a_i = b_i + max\{t - a_i, 0\}$

$\boldsymbol{g(S,t) = min\{a_i + g(S-\{i\}, b_i + max\{t-a_i, 0\})\}}$ **(Eqn 1)** ,where i belongs to S.

Now, let R be any schedule for a job subset S. Let i and j be the first two jobs.

$g(S, t)$= length of an optimal schedule for the subset of jobs S under the assumption that P_2 is not available until time t.

Thus, for $S=\{1, 2,..n\}$,

$g(S,t)=g(\{1,2,..n\},0)$

$\boldsymbol{g(\{1,2,\ldots n\},0) = min\{a_i + g(\{1,2,\ldots n\}-\{i\}, b_i)\},}$ where $1 \leq i \leq n$

$g(S,t) = a_i + g(S-\{i\}, b_i + max\{t - a_i, 0\})$

$\mathbf{g(S,t) = a_i + a_j + g(S-\{i,j\}, b_j + max\{b_i + max\{t-a_i,0\} - a_j,0\}}$ **(Eqn 2)**

Above equation can be simplified as,

$t_{ij} = b_j + max\{b_i + max\{t-a_i,0\} - a_j,0\}$

$= b_j + b_i - a_j + max\{max\{t-a_i,0\}, a_j - b_i\}$

$= b_j + b_i - a_j + max\{t-a_i, a_j - b_i, 0\}$

$t_{ij} = b_j + b_i - a_j - a_i + max\{t, a_i + a_j - b_i, a_i\}$

If jobs *i* and *j* are interchanged in R, then

$\mathbf{g'(S,t) = a_i + a_j + g(S-\{i,j\}, t_{ji})}$ **(Eqn 3)**

where, $t_{ji} = b_j + b_i - a_j - a_i + max\{t, a_i + a_j - b_j, a_j\}$

$\mathbf{max\{t, a_i + a_j - b_i, a_i\} \leq max\{t, a_i + a_j - b_j, a_j\}}$ **(Eqn 4)**

comparing, eqns. (1)and(3) , if eqn. (4) holds, then

$g(S,t) \leq g'(S,t)$

→ There exists an optimal schedule in which for every pair(i, j) of adjacent jobs,

$\min\{b_i, a_j\} \geq \min\{b_j, a_i\}$, holds true.

→ For making an optimal schedule,

If $\min\{a_1, a_2, \ldots a_n, b_1, b_2, \ldots b_n\}$ is a_i, then job i should be the first job.

If $\min\{a_1, a_2, \ldots a_n, b_1, b_2, \ldots b_n\}$ is b_j, then job j should be the last job.

→ The above procedure helps positioning one of the n jobs and we need to reapply the eqn.(5) to position the remaining n-1 jobs.

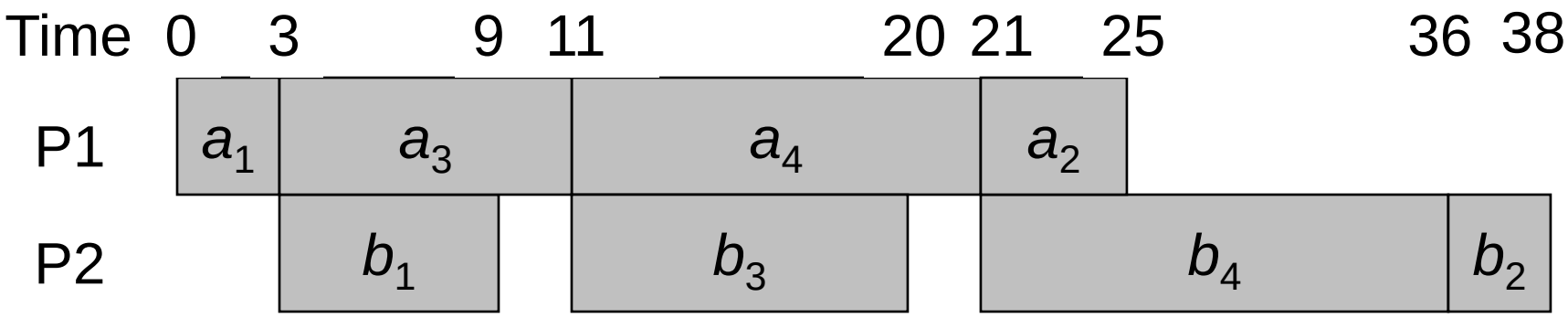
Johnson's Scheduling Rule

→ Sort all the a_i's and b_j's into non-decreasing (i.e. ascending) order.

→ If the next number in the sequence is a_j and job j hasn't yet been scheduled, schedule job j at the leftmost available spot. If the next number is b_j and job j hasn't yet been scheduled, schedule job j at the rightmost available spot. If j has already been scheduled go to the next number in the sequence.

Example: Let $n = 4$, $(a_1,a_2,a_3,a_4)=(3,4,8,10)$ and $(b_1,b_2,b_3,b_4) = (6,2,9,15)$. Now sort all these tasks into non-decreasing order:

$(b_2,a_1,a_2,b_1,a_3,b_3,a_4,b_4) = (2,3,4,6,8,9,10,15)$. Let $\sigma_1,\sigma_2,\sigma_3,\sigma_4$ be the optimal schedule. Since the smallest number is b_2, we set $\sigma_4 = 2$.



Time Complexity

→ Solving eqn.(1),

$$g(S,t) = min\{a_i + g(S-\{i\}, b_i + max\{t-a_i, 0\})\} \quad \text{(Eqn 5)}$$

directly for $g(\{1,2,\ldots n\},0)$ for the optimal schedule will take $\Omega(2^n)$.

→ General flow shop scheduling problem with $m > 3$ is *NP-hard* problem whereas the one minimizing the makespan(with $m = 2$) is *NP-complete*.

→ The scheduling rule discussed in previous slide(Johnson's Rule) can be implemented to run in time $O(n \log n)$. Johnson's rule is a greedy algorithm that optimizes the time complexity for the special case of $m = 2$.

Thank You!
Any Queries?